

Distributed machine learning workloads and high-performance computing clusters increasingly rely on barrier-synchronized collective communication operations such as AllReduce, Ring-Shift, and Gather. In these workloads, participating workers must complete their communication before computation can proceed, making the completion time of an operation sensitive to its slowest participant. Consequently, transient network delays affecting a single participant can become operation-level stragglers and directly extend job execution time. Despite this dependency, network telemetry systems predominantly observe packets and flows independently, providing limited visibility into the performance of an entire collective operation.

Traditional network telemetry, particularly In-Band Network Telemetry (INT), embeds switch-level information such as timestamps, queue occupancy, and hop latency into packets. Recent work has explored reducing the resulting overhead through selective telemetry, information-aware reporting, probe optimization, and traffic-aware telemetry activation. For example, P4InfoSen-INT allows programmable switches to selectively insert telemetry according to the information content of measurements, while recent systems such as INT-LLPP and FANT reduce monitoring overhead through selective probe collection and adaptive telemetry mechanisms. These approaches, however, primarily optimize **which network telemetry should be collected**; they do not exploit the synchronization relationship among flows belonging to the same distributed operation.

Recent work has also considered straggler detection directly in programmable data planes. STRAGFLOW, for example, detects packets experiencing excessive queueing delay at line rate and reports the affected flows to the control plane. However, its detection abstraction remains flow-centric: a packet or flow is identified as a straggler based on its individual network behavior. At the application level, systems such as NCCL telemetry and Google's CoMMA can associate measurements with collective operations and identify communication stragglers, but the telemetry is generated from the collective communication runtime rather than being aggregated within the network data plane. Thus, there remains a gap between **network-level runtime observation** and **collective-level synchronization semantics**.

We address this gap through an **in-switch, collective-aware telemetry aggregation framework** implemented in programmable data planes. Rather than treating participating packets as independent 5-tuple flows, the proposed design introduces a lightweight context shim that associates flows belonging to the same collective operation with a shared operation context identifier. Programmable switches use this context to maintain compact state for the active synchronization step, allowing telemetry to be aggregated across participating flows directly in switch memory.

For each operation context, the switch maintains the minimum and maximum observed arrival timestamps in stateful P4 registers and derives the intra-operation temporal skew, $\delta t = t_{\text{slow}} - t_{\text{fast}}$. The switch continuously compares this skew against an operator-defined threshold. Normal packets therefore contribute only to local aggregated states and generate no telemetry reports. When the observed skew exceeds the threshold, the switch identifies the lagging participant and selectively clones a compact telemetry report containing the participant identifier

and relevant local network state, such as queue occupancy. The result is a **collective-aware, event-driven telemetry mechanism** that shifts telemetry processing from per-packet reporting to **in-network operation-level aggregation and straggler-triggered reporting**.